History-Conditioned SAC under Sensor Faults: A Pilot Study of Adaptation and Retention

~~Anonymous Author(s)~~Felix Surjodinoto , Keandre Rafael , Delroy , and Samuel Philip

Computer Science Department, School of Computer Science,
Bina Nusantara University, Jakarta 11530, Indonesia
{felix.surjodinoto,keandre.rafael,delroy,samuel.philip}@binus.ac.id

Abstract. ~~Adapting during training while retaining earlier behavior requires measuring both outcomes . We report a warm-start pilot of history-conditioned~~ A controller that keeps learning after its sensing changes should improve under the new condition without losing what it could already do, so both outcomes must be measured. We study continued Soft Actor-Critic ~~under incorrect or delayed observations in HalfCheetah-v5. Three pretrained transformer policies and three reactive policies each branch into~~ training of a history-conditioned transformer and a reactive controller after the onset of observation noise or a three-step ~~delay~~observation delay in HalfCheetah-v5 and Ant-v5. Eight source checkpoints per controller and task each branch into the two faults, with or without half of each batch drawn from earlier clean experience, plus a control that continues on clean sensing. Every branch runs 20,~~with either no retained replay or a 50% mixture of earlier clean-condition experience. All 24 branches receive 4~~,000 ~~new-condition interactions and matched update counts. The same updated model is evaluated on both the sensor condition and the original clean condition. Without retained replay, mean delay-condition return improves by 98.1 for the transformer and 383.7 for the reactive controller , while clean-condition return declines by 825.4 and 1,158.9, respectively. With retained replay , those~~ steps, and the same weights are evaluated under the fault and under clean sensing. In HalfCheetah, the reactive controller improved under delay (+689) while its clean return fell (−1,390); the transformer did not improve under delay and lost clean return (−2,496). There, retained replay reduced each clean-condition ~~declines are 145.2 and 383.9, but delay-condition gains fall to 10.2 and 228.2. Mean noise-condition performance declines in all four controller/replay combinations. Every branch loses measured clean-condition performance. Saved trajectories verify actual observation corruption, and checkpoint audits verify paired initialization and same-model retention measurement. These short-budget,~~ loss, with all eight seeds agreeing, but shrank the reactive controller's delay gain, and an exploratory critic diagnostic shows that under delay, mixing kept the critics' values on delayed histories inflated relative to clean ones. The Ant sources learned to stand rather than walk, and the faults cost them little. Inference took 0.58 ms per action for the transformer on one CPU thread. The study reproduces an earlier three-seed ~~results expose an adaptation–retention tradeoff in the tested configuration; they do not establish fast adaptation~~

~~without forgetting or a general advantage of history conditioning~~pilot exactly but does not confirm its small transformer gain under delay.

Keywords: Reinforcement learning · Sensor faults · History conditioning · Retention · Experience replay

1 Introduction

An adaptive controller should learn to operate under new conditions while preserving ~~useful~~ behavior acquired earlier. ~~Rapid improvement on a new task and retention of an old task~~ These are separate requirements: a policy can improve on ~~one~~ a new condition while deteriorating on ~~the other~~an old one, and a training plateau is not the same as forgetting. Our long-term ~~research objective is to combine these capabilities in~~ objective is real-time ~~control. During development of an SAC-based controller, training plateaus in experiments~~ visual control in TrackMania 2020 (Fig. 1a), where training plateaus with transformer and world-model-~~style~~components motivated a ~~more controlled investigation of how recent~~ controlled study of how interaction history and retained experience affect ~~adaptation. Those development observations motivate this study; they are not a matched architectural comparison or evidence of a diagnosed cause of the plateaus.~~

~~A plateau must also be distinguished from forgetting. A model that stops improving has not necessarily lost previously acquired behavior. Conversely, continued improvement on current data can conceal deterioration on earlier conditions. Work on primacy biasidentifies an influence of early experience on later reinforcement learning (RL)~~continued learning. Primacy bias [13], ~~while research on plasticityexamines a model's continuing ability to learn [4] . These provide relevant context, but neither mechanism~~ loss of plasticity [4] and forgetting during fine-tuning [18] are relevant mechanisms, but none can be assigned to a ~~particular stalled run without an appropriate measurement. We therefore measure~~ run without measuring new-condition ~~performance~~ and old-condition ~~retention directly, rather than infer either from training loss.~~ performance directly. We therefore measure both.

The ~~intended intervention here~~ intervention is a change in ~~sensing~~*sensing*. A sensor fault alters what the policy observes~~: a reading may be noisy or may describe an earlier state. An actuator fault instead~~ , whereas an actuator fault alters the applied control~~. These interventions can create different information requirements even when both reduce return. An earlier benchmark in our development process altered actions; inspection revealed that its sensor-noise label did not implement observation corruption. The present experiment uses a separately verified observation wrapper, and the earlier actuator results are not relabeled as sensor evidence.~~

; delayed or noisy signals change the information a policy needs [17]. History conditioning supplies ~~a policy with a sequence of~~ past observations, actions ~~, and rewards. A~~ and rewards; a transformer is one way to encode ~~that information,~~

~~while~~ them, and Soft Actor-Critic (SAC)~~supplies a learning objective [8]. These are complementary design choices.~~ [8] supplies the learning objective. In-context RL uses history to ~~change behavior~~ adapt with fixed parameters~~, as illustrated by RL^2 [5] and subsequent sequence-based methods [6,11]. Our present~~ [5,6,11]. Our question concerns *continued parameter learning*: after a trained controller encounters changed sensing, does further training improve its return, ~~and~~ what happens to its earlier clean-condition performance, and what does mixing earlier clean experience into replay do?

We ~~conduct an explicitly bounded pilot~~ study this in HalfCheetah-v5 ~~. Three existing checkpoints per architecture~~ and Ant-v5. Eight source checkpoints per controller and task initialize a history-conditioned transformer and a reactive controller. Each checkpoint branches into two sensor ~~conditions~~ faults and two replay ~~strategies. Both strategies receive the same new-condition interaction and update budgets; one also samples a fixed pool of earlier clean-condition experience. We evaluate~~ fractions, plus a control that continues training on the clean condition. All branches receive 20,000 new interactions, and the same updated ~~model~~ weights are evaluated on the new and ~~old conditions at fixed budgets. The resulting 24 branches are derived from six source models, not 24 independent training seeds~~the old condition. The study extends an earlier three-seed, 4,000-step pilot, whose branches it reproduces exactly; the statistical analysis was fixed before the runs.

~~This study contributes~~ Our contributions are (i) ~~an implemented and~~ a behaviorally verified sensor-fault intervention for continued SAC training of history-conditioned and reactive controllers; (ii) ~~a paired continued-training comparison with explicit~~ paired adaptation and retention measurements ~~; and~~ in two environments, with a control that continues training on clean sensing and with pre-specified paired tests; (iii) ~~an auditable record of the experiment's limits. It does~~ a critic-level analysis of replay mixing based on held-out soft Bellman residuals and value bias; and (iv) inference-latency measurements and released code, wrappers, seeds and checkpoints. We do not propose a new ~~general-purpose algorithm , establish adaptation to arbitrary new environments, or equate a short pilot with a~~ algorithm or claim a solution to forgetting.

1.1 Related Work

Memory, delay and adaptation. Recurrent model-free policies ~~can be strong baselines in partially observed tasks when their design and training are carefully controlled~~are strong baselines under partial observability when carefully tuned [12]. ~~AMAGO studies off-policy transformer-based~~ Off-policy transformer agents for in-context ~~RL [6],~~ and multi-task RL are studied in AMAGO and AMAGO-2~~addresses multi-task learning across differing return scales [7].~~ [6,7], and Algorithm Distillation learns from ~~algorithm-generated~~ learning histories [11]. ~~Our much smaller configuration uses a conventional critic-regression objective and directly collected experience. It does not reproduce those complete methodsor evaluate their task breadth. Its history resets between episodes; continued parameter updates supply the~~ Signal delay degrades standard deep RL, and delay-aware

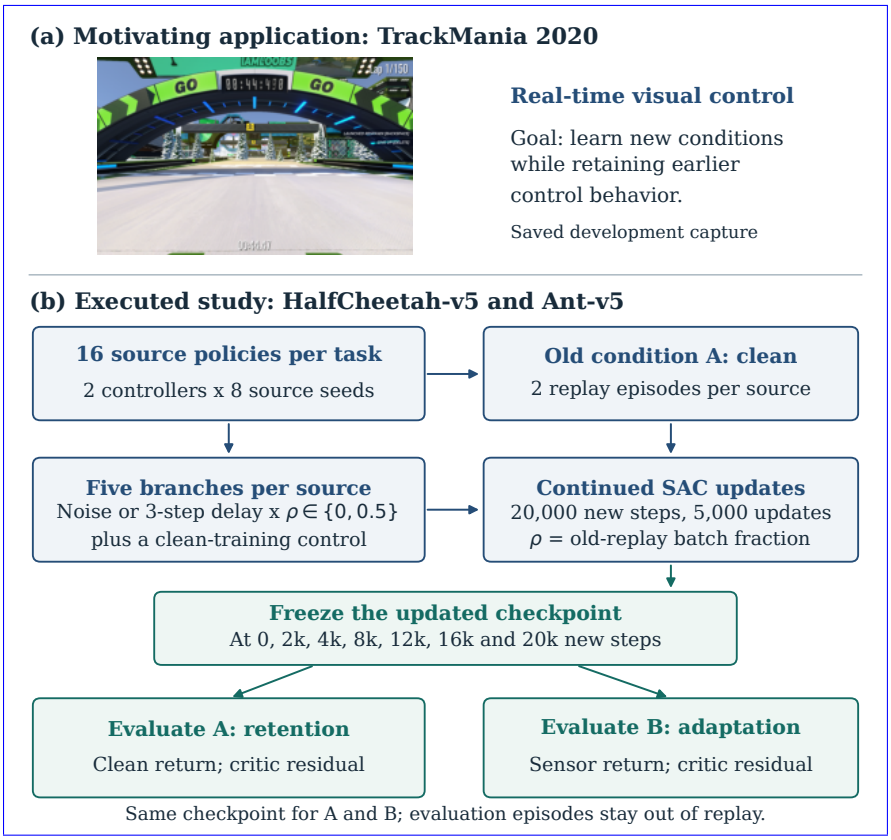


Fig. 1. Research context and workflow. (a) A TrackMania 2020 development capture from a camera-pipeline preflight; the game clock is not a benchmark lap time. (b) The executed MuJoCo procedure. Each source policy branches into five continued-training runs; old and new conditions are evaluated with the same frozen weights. All quantitative results come from panel (b).

state augmentation and prediction recover much of the loss [17]. We use none of these complete methods; our transformer is a small SAC encoder whose history resets each episode, so cross-episode adaptation ~~examined here~~happens only through parameter updates.

Replay and retention. CLEAR combines replay with ~~additional learning objectives to address stability and plasticity in~~ behavioral cloning for continual RL [15]~~. RLPD studies~~, RLPD uses prior data for efficient online RL~~using prior data [1]. These findings motivate testing retained experience, but they do not imply that an arbitrary old/new replay mixture prevents forgetting. Our intervention changes the source of~~ [1], and fine-tuning pre-trained RL models forgets capabilities in states the new task does not visit early, which knowledge-retention techniques

mitigate [18]. Our retained replay is the simplest rehearsal: half of each ~~update batch . It does not implement CLEAR's behavioral-cloning objective or compare against a full RLPD implementation. We report its effect on both evaluation conditions.~~ batch is old clean experience, with no cloning or feature-preservation loss.

Optimization and empirical design. The ~~pilot inherits~~ controllers use a REDQ-style critic ensemble [3]~~from the existing SAC controllers.~~ ; DroQ and CrossQ ~~illustrate~~ show that normalization and update ~~scheduling can affect learning~~ schedules affect efficiency [9,2]. ~~Primacy bias and loss of plasticity provide further reasons to examine the complete learning process [13,4]. None of these mechanisms is separately ablated here. Following~~ Benchmark results with few seeds show *that* a method performed well rather than *why* [10], and empirical-design ~~principles [14] , we fix budgets, retain all specified seeds , expose the initial performance of each branch, and distinguish descriptive seed variation from a formal test of equivalence or no forgetting~~guidance calls for fixed budgets, all seeds and appropriate uncertainty [14]. We follow both by pairing outcomes with controlled contrasts and a mechanism-level critic diagnostic.

2 Method

2.1 Environments and Sensor ~~Intervention~~Faults

We use the Gymnasium MuJoCo tasks [16] HalfCheetah-v5 ~~through Gymnasium [16], with~~ (17 observation coordinates~~, six continuous control dimensions, and a~~ : 8 positions and 9 velocities; 6 actions; no termination) and Ant-v5 (27 coordinates without the 78 contact forces; 13 positions and 14 velocities; 8 actions; terminates when the torso leaves the healthy height range). Episodes last at most 1,~~000-step episode horizon. The default observation contains eight position coordinates , excluding absolute horizontal position, followed by nine velocities~~000 steps and actions lie in $[-1, 1]$. ~~The action bounds are $[-1, 1]$ in each dimension.~~ Condition A uses ~~unmodified observations. Two separate choices of condition~~ true observations. Condition B ~~alter sensing while preserving the requested control and base reward function.~~

alters only the sensor vector; commands and the base reward are unchanged. For noise, the policy receives

$$o_t = s_t + \epsilon_t, \qquad \epsilon_{t,j} \sim \mathcal{N}(0, \sigma_j^2), \tag{1}$$

independently across coordinates and steps, ~~where~~ with $\sigma_j = 0.1$ for ~~the eight position coordinates and $\sigma_j = 0.5$ for the nine velocities . These values are in native observation units; no normalizer is fitted using evaluation data. A separate seeded generator produces the corruption so that sensor draws do not change the environment~~positions and 0.5 for velocities in native units, drawn from a generator separate from the simulator's reset randomness.

For delay, ~~the policy receives~~ $o_t = s_{t-3}$~~. Before three prior steps are available, the sensor vector is zero~~ . ~~The FIFO queue resets at each episode boundary. Both sensor conditions begin at episode~~ , with a zero vector until three earlier readings exist. Both faults start at reset and remain active~~throughout the episode. They are known, fixed study conditions; this pilot is not a meta-test over a previously unobserved distribution of sensor mechanisms or severities.~~ .

~~The model input is~~

$$~~x_t = [o_t, a_{t-1}, r_{t-1}, d_{t-1}] \in \mathbb{R}^{25}.~~$$

A Gymnasium wrapper implements both faults. At each step it forwards the command unchanged to the simulator, appends the true observation to a four-element FIFO that is cleared at reset, and emits

$$x_t = [o_t, a_{t-1}, r_{t-1}, d_{t-1}] \in \mathbb{R}^{25} \text{ (HalfCheetah) or } \mathbb{R}^{37} \text{ (Ant)}, \tag{2}$$

~~The command and reward fields retain their current timing; only the base sensor vector is corrupted or delayed. All preceding-action, reward, and termination fields~~ where a_{t-1}, r_{t-1} and the termination flag d_{t-1} keep their correct timing and are zero at reset. ~~The final flag denotes environment termination rather than time-limit truncation. True sensor readings are saved for verification but are excluded from the policy input and loss. Thus, the experiment concerns faults in the state-observation channel, not simultaneous corruption of every information source.~~ True readings are logged for audits only. Under delay, s_t is in principle recoverable from $o_t = s_{t-3}$ and the last three commands, because the dynamics are deterministic [17]; a policy that sees only x_t has a_{t-1} but not a_{t-3} and a_{t-2}.

2.2 ~~Source Policies and Context Support~~

~~The source models are the validation-selected checkpoints for training seeds 42–44 from an earlier actuator-perturbation study. That study trained on action scaling and action noise; it did not train on these observation faults. The present experiment does not reconstruct those historical training runs. Source checkpoint hashes and the compatible model definitions are captured before this pilot. Condition A is a clean evaluation condition on which these pretrained models already have measurable performance; it is not the result of a new clean-only pretraining phase.~~

~~The transformer maps each packed token through a multilayer embedding with LayerNormand GELUinto~~

2.2 Controllers and History Conditioning

Table 1 summarizes both controllers. The transformer receives the most recent at most 32 tokens of the current episode. Each token passes through a two-layer embedding (Linear, LayerNorm, GELU, Linear; 64 features~~. Two causal transformer~~

Table 1. Controller architectures (HalfCheetah sizes; Ant differs only in input and action width). Parameters exclude target networks.

Component	Transformer / reactive	Parameters
Encoder	2 causal blocks, 2 heads, width 64, FF 128, $\leq$32 tokens, linear z_t head / none	81,736 / 0
Actor	$[h_t, z_t] \to 2\times128$ / $x_t \to 2\times256$; tanh-Gaussian	27,404 / 75,532
Critics	10 heads, 2×256 with LayerNorm, on $[h_t, z_t, a]$ / $[x_t, a]$	872,970 / 752,650

~~blocks use two attention heads each and a feed-forward width of 128. Learned absolute positional embeddings provide 128 positions. An eight-dimensional task embeddingis concatenated with the transformer output. The actor has two 128-unit hidden layers; each of ten criticheads has two 256-unit hidden layers with LayerNorm. A classifier inherited from the source model is unused: there is no auxiliary fault-label loss in this pilot.~~

~~The reactive baseline has a two-layer, 256-unit Gaussian actor and ten two-layer, 256-unit LayerNorm~~) plus a learned absolute position embedding, then two pre-norm causal transformer blocks. The output at the newest token, h_t, and an 8-dimensional linear projection z_t of it form the 72-dimensional feature used by the actor and by every critic. The context resets at each episode, is recomputed at every step without caching, and carries no information across episodes; only parameter updates do. The reactive controller maps the single token x_t directly to its actor and critics. It ~~receives the same 25-dimensional token from Eq. 2. Consequently, it retains access to the~~ therefore sees the previous command and reward ~~even though it has no accumulated sequencestate. Architectural~~ but cannot integrate a sequence. The controllers differ in capacity and preprocessing ~~differ; comparisons between the two controllers do not isolate memory as a single causal factor~~as well as memory, so comparisons between them are controller comparisons, not isolated causal estimates of memory.

~~The earlier transformer implementation trained on 32-token windows but could collect and evaluate with~~

The source models for each task are validation-selected checkpoints of training seeds 42–49 from one SAC recipe: 100 iterations, each collecting four episodes under randomly sampled *actuator* perturbations (action scaling and action noise) and then performing 100 actor and temperature updates, each preceded by ten critic-ensemble updates (at most 400,000 steps; 100,000 critic and 10,000 actor updates). Transformer batches hold 64 ~~tokens. The new pilot restricts collection and evaluation to a rolling queue of at most 32 tokens before any initial or final measurement. Replay reconstructs exactly the corresponding current and successor histories for each sampled transition. A full successor context slides by one token,while an episode-start context grows until it reaches the cap. Right padding follows all valid tokens; causal attention prevents the queried token from attending to that padding.~~ sequences of 24 steps whose first 8 steps only build context; reactive batches hold 64 transitions. The transformer recipe adds

a fault-classification loss (weight 0.05) whose head is unused here. No source trained with sensor faults.

Each sampled history supervises its final valid transition. With probability 0.25,the replay sampler selects a transition from the first up-to-32 transitions of an episode; otherwise it selects uniformly over the episode. Final transitionsare eligible, and an actual successor observation is retained for time-limit bootstrapping. These choices support both short deployment contexts and mature rolling histories. Positions beyond 31 remain unused. The experiment therefore changes the sequence-training implementation as well as the environmental intervention; it is not a replication of the earlier result under a new label.

2.3 Paired Adaptation and Continued Training, Retained Replay and Control

For each source model, we first collect two deterministic clean-condition episodes , giving 2,000 old-condition transitions. This fixed pool is shared by all descendant branches of that model. It is distinct from the evaluationepisodes. Both replay variants incur the same old-data collection; only the retained-replay variant uses the pool for updates.

two deterministic clean episodes form a fixed old pool shared by its branches and disjoint from evaluation. Each source model then initializes four independent five branches: noise or delay , crossed with old-replay fraction $\rho \in \{0, 0.5\}$. Optimizers and entropy temperature are reset identically at each branch because their historical states were not stored in the source checkpoints. Parameters are updated during adaptation. A branch collects four stochastic-policy episodes in condition B,totaling 4., and a control that continues training on A with $\rho = 0$. The control measures the change caused by continued optimization alone. Each branch collects 20,000 new-environment steps . After each episode it performs 250 steps with its stochastic policy and performs 0.25 critic-ensemble and actor update cycles. A batch has updates per step (5,000 each) on batches of 64 supervised transitions: all new-condition samples new for $\rho = 0$, or exactly 32 old and 32 new samples for $\rho = 0.5$. Sampling uses replacement from the available episode pools. The same Collection is step-budgeted, so terminated Ant episodes restart and episodes are cut at evaluation budgets (as time-limit truncations); replay draws episodes in proportion to their length. Paired branches share their adaptation seed and environment-reset scheduleare used for paired replay variants; their trajectories can diverge as their policies changereset schedule. Optimizers and the entropy temperature are re-initialized identically because they were not stored with the sources.

For each critic update, the target uses the current actor's next action and the minimum of two randomly selected target critics from the ensemble of ten . The discount is 0.99. Critic The critic target is $y = r + \gamma(1 - d)\big(\min_{j \in S} \bar{Q}_j(h', a') - \alpha \log \pi(a'|h')\big)$ with $a' \sim \pi(\cdot|h')$, S two random critics of ten and $\gamma = 0.99$; critic losses are summed over the ensemble. The actor minimizes the entropy-adjusted negative value using critics 1 and 2. Transformer features are detached for the actor update; critic regressionupdates the shared representation. Target encoders and

Table 2. Continued-training configuration (both tasks). Budgets exclude source training.

Setting	Value
Source seeds per controller and task	8 (42–49)
Branches per source model	noise/delay $\times$ $\rho \in \{0, 0.5\}$, clean control
New-condition steps; updates per branch	20,000; 5,000
Clean replay pool per source model	2 episodes
Batch size; ensemble critics	64; 10
Actor / critic / temperature learning rates	10^{-4} / 10^{-4} / 3×10^{-4}
Initial temperature; target entropy	0.1; $-\lvert\mathcal{A}\rvert$
Discount; target retention; gradient clip	0.99; 0.995; 10
Evaluation budgets (steps)	0, 2k, 4k, 8k, 12k, 16k, 20k
Evaluation episodes per condition and budget	5 deterministic, fixed seeds

~~critics~~ $\alpha \log \pi - \min(Q_1, Q_2)$ on detached transformer features, so the representation learns through critic regression. Target networks use Polyak retention 0.995 ~~. Dropout is disabled consistently in collection, evaluation, and fine-tuning; this does not disable gradient computation. No fault identifier is used for training.~~

~~There is one critic-ensemble update per actor update and 0.25 of each per newly collected environment step. The pilot therefore differs from the original source-model update schedule. Each transformer replay sample trains one valid transition , rather than all positions of a sequence; its context still carries a computational cost~~and dropout is disabled throughout. Each transformer sample trains its final valid transition with the exact rolling history that produced it; one quarter of samples come from the first 32 steps of an episode, so short contexts are represented. Table 2 ~~records~~ lists the settings. ~~The run environment is Python 3.12.10, PyTorch 2.11.0 with CUDA 12.8, Gymnasium 1.3.0, and MuJoCo 3.12.0 on Windows with an NVIDIA GeForce RTX 5070 Laptop GPU. Action inference is on CPU and parameter updates are on GPU.~~

2.4 ~~Adaptation and Retention~~ Measurements ~~At budgets $n \in \{0, 2000, 4000\}$, we evaluate both A~~ and ~~B using~~ Statistical Analysis

At each budget n, the same checkpoint ~~. Parameters are frozen during evaluation, and context resets each episode . Every condition uses three matched environment seeds , $860000 + 100e$ for $e = 0, 1, 2$. These episodes~~ is evaluated on A and B with frozen parameters, a fresh context per episode and five fixed environment seeds that never enter replay. ~~Evaluation RNG state is isolated from the training RNG. Checkpoints are retained at fixed budgets; neither sensor performance nor retention determines checkpoint selection~~For source seed i, fault c and fraction ρ, the new-condition change is $\Delta^B(n) = R^B(n) - R^B(0)$ and the retention change is $\Delta^A(n) = R^A(n) - R^A(0)$, where R is the mean evaluation return. A negative

Δ^A is a loss of measured clean performance. Checkpoints are fixed by budget; no outcome selects them.

~~Let $R^A_{i,c,\rho}(n)$~~ The replication unit is the source seed ($n = 8$); evaluation episodes are averaged within a seed first, and branches from one source model are not independent. The analysis was fixed before the runs. The primary family is the paired replay effect $D_i = \text{metric}_i(\rho{=}0.5) - \text{metric}_i(\rho{=}0)$ for Δ^A and ~~$R^B_{i,c,\rho}(n)$ denote mean evaluation returns for source seed i, sensor condition c, replay fraction ρ,and adaptation budget n~~Δ^B at 20,000 steps, for each controller and fault (eight tests per task), using exact two-sided Wilcoxon signed-rank tests with Holm correction. We report

~~$$\Delta^B_{i,c,\rho}(n) = R^B_{i,c,\rho}(n) - R^B_{i,c,\rho}(0),$$~~

~~$$\Delta^A_{i,c,\rho}(n) = R^A_{i,c,\rho}(n) - R^A_{i,c,\rho}(0).$$~~

~~A positive new-condition change indicates improved return within the tested budget. A negative old-condition change indicates a loss in measured clean-condition performance. Retention always uses the updated model; restoring the earlier checkpoint would invalidate that comparison.~~ mean differences with 95% t intervals and seed-bootstrap intervals (10,000 resamples). A secondary family tests fault-specific forgetting, Δ^A of each $\rho = 0$ fault branch minus Δ^A of the clean control (four tests per task); per-arm changes are tested against zero in the released analysis files. With eight seeds the smallest attainable exact p is 0.0078, so a Holm-adjusted p cannot fall below 0.0625 in the eight-test primary family or below 0.031 in the four-test family. This floor follows from the pre-specified design, and we report it rather than change the correction after the runs: conclusions about the replay effect rest on the interval estimates and on sign agreement across seeds, and non-significance is not evidence of equivalence.

~~We also compute a trapezoidal average of $R^B(n)$ over the sampled 0–4,000-step learning curve and paired differences between replay variants. Adaptation elapsed time includes collection and optimization but excludes evaluation and checkpoint I/O. No recovery threshold or acceptable retention-loss margin was specified, so we do not report time to recovery or claim equivalence to the old performance. The three training seeds, rather than their repeated evaluation episodes or descendant branches, are the replication units. We retain every seed and report descriptive means and ranges. Shaded ranges in figures are not confidence intervals.~~

2.5 Critic Diagnostic for Replay Mixing

With fraction ρ, the critic regresses onto targets drawn from the mixture $(1-\rho)\mathcal{D}_B + \rho\mathcal{D}_A$. Two distortions are possible. Old transitions were generated by the source policy, so their targets rely on the current actor's actions in histories it is not being trained to act in (an off-policy mismatch). Moreover, a history can occur under both sensing channels while implying different states, especially under delay; its regression target is then a ρ-weighted blend of two Bellman targets, biasing the fitted value for both. To measure these effects, we evaluate each saved

checkpoint's critics on its own held-out evaluation trajectories from A and B. We report the root-mean-square soft Bellman residual $y - \frac{1}{10}\sum_j Q_j(h,a)$, computed with the training target and the saved temperature, averaging over all critic pairs, and the Monte-Carlo bias $Q - G_t$ for steps with a complete discounted return G_t. The critics estimate the soft value of the stochastic policy whereas G_t is the deterministic policy's return. This offset is shared by the A and B data of one checkpoint, so instead of raw biases we compare the value-bias gap, the bias on B data minus that on A data.

2.6 ~~Implementation~~ Verification, Latency and ~~Evidence Capture~~Reproducibility

~~Eleven~~ Twelve behavioral tests check ~~sensor~~ corruption, timing, random-stream separation, replay boundaries, masking, parameter updates ~~,~~and positional support. A real MuJoCo test applies identical commands under clean, noisy ~~, and delayed observations~~ and delayed sensing and requires identical ~~underlying~~ states and rewards. ~~This verifies that the intervention acts on sensing. A separate short execution check exercises all eight architecture/fault/replay combinations using reduced horizons; those returns are excluded from the pilot analysis.~~

~~The pilot snapshots its exact~~ Every run snapshots its source files, ~~configuration~~protocol, package versions and checkpoint hashes, and ~~input hashesbefore collection. Every evaluation~~ records commands, ~~model-input observations~~model inputs, audit-only true ~~observations, sensor ages, rewards , and checkpoint hashes. An independent readback recomputes returns, checks that delayed readings match the recorded earlier states, verifies paired initial performance, and confirms that the same model was used for A and B at each budget. Original source-checkpoint hashes are checked again at completion. This captures the new experiment's provenance without claiming to reconstruct the incomplete historical source capture of pretraining.~~ readings and rewards for every evaluation. We measure inference latency of the deployment call (a full 32-token forward pass for the transformer) at batch size one with real model inputs, excluding simulation, on CPU with one and four threads and on GPU. Runs use Python 3.12.10, PyTorch 2.11.0 with CUDA 12.8, Gymnasium 1.3.0 and MuJoCo 3.12.0 on Windows with an NVIDIA GeForce RTX 5070 Laptop GPU; rollouts infer on CPU and updates run on GPU.

3 Results

3.1 ~~Completed Runs~~ Reproducibility and ~~Initial Performance~~Executed Budgets

All ~~24 specified branches completed the fixed budget. Separate readback verified 432 evaluation episodes and 48 saved candidate checkpoints,with the six original source checkpoints unchanged. Paired replay variants produced identical initial evaluation returns . The complete study, including clean replay collection,evaluation, checkpoint writing, and initialization,took 1, 463.8 seconds on the recorded~~

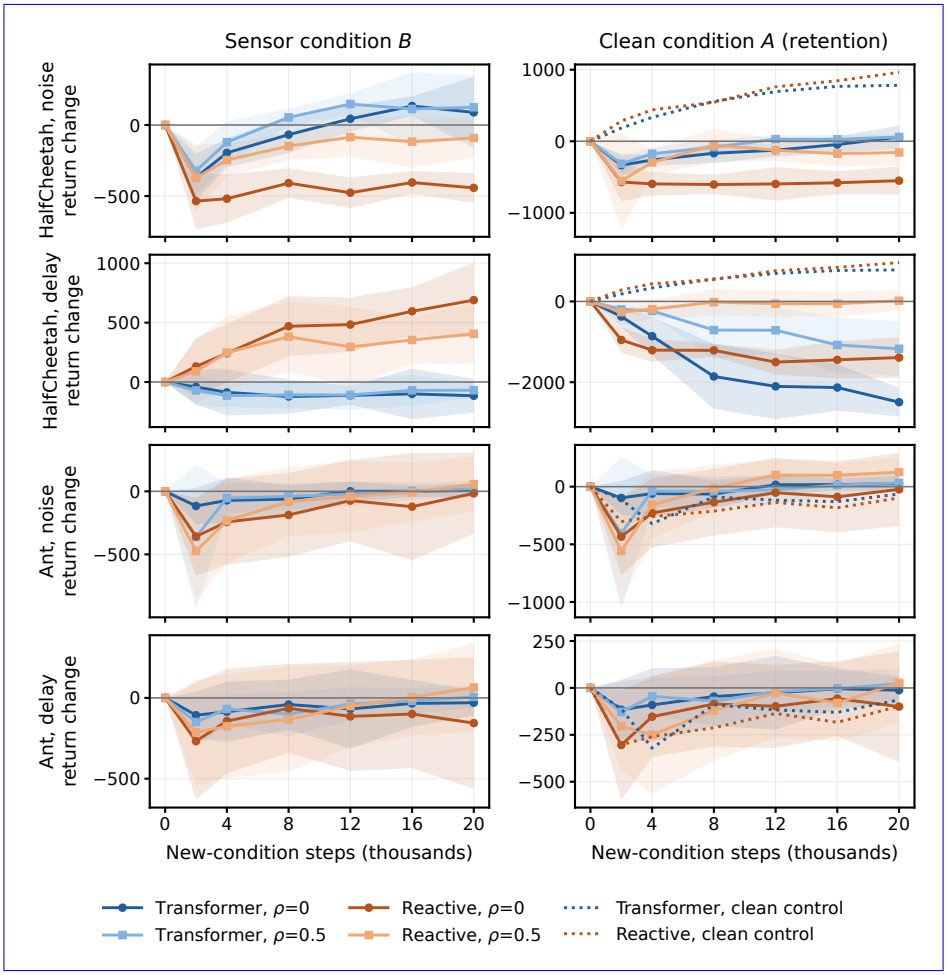


Fig. 2. Mean change in sensor-condition return (left) and clean-condition return (right) during continued training in HalfCheetah (top two rows) and Ant (bottom two rows), with 95% t intervals over eight source seeds. Dotted lines are the clean-continuation controls.

~~machine. Table 3 gives the equally weighted mean over the three source seeds; returns are environment reward sums, not completion rates~~80 HalfCheetah and 80 Ant branches ran exactly 20,000 steps and 5,000 updates. The 24 HalfCheetah branches that share their seeds with the pilot reproduce its returns at 0, 2,000 and 4,000 steps exactly (largest absolute difference 0.000), and recomputing the pilot's metric from them returns its published means. After a memory failure, 9 completed reactive branches were rerun from scratch; their evaluation returns and 9 of 9 final checkpoints are bit-identical to the first run.

Table 3. Outcomes at 20,000 steps: mean over eight source seeds with 95% t intervals. $B(0)$ is the zero-step sensor-condition return; ΔB and ΔA are the changes in sensor- and clean-condition return. For the clean control, $B = A$.

Controller	Fault	ρ	$B(0)$	ΔB [95% CI]	ΔA [95% CI]
HalfCheetah-v5					
Transformer	Noise	0	2,402	+89 [−163, 341]	+59 [−104, 222]
Transformer	Noise	0.5	2,402	+125 [−96, 346]	+59 [14, 104]
Transformer	Delay	0	13	−115 [−259, 28]	−2,496 [−2,852, −2,139]
Transformer	Delay	0.5	13	−69 [−130, −9]	−1,173 [−1,846, −500]
Transformer	Clean control	0	2,695	—	+782 [606, 958]
Reactive	Noise	0	2,900	−443 [−546, −339]	−550 [−748, −352]
Reactive	Noise	0.5	2,900	−91 [−230, 47]	−157 [−339, 26]
Reactive	Delay	0	−179	+689 [373, 1,006]	−1,390 [−1,896, −884]
Reactive	Delay	0.5	−179	+406 [153, 658]	+16 [−233, 266]
Reactive	Clean control	0	3,239	—	+963 [740, 1,187]
Ant-v5					
Transformer	Noise	0	921	+7 [−46, 61]	+27 [−37, 90]
Transformer	Noise	0.5	921	+22 [−40, 85]	+34 [−38, 105]
Transformer	Delay	0	918	−30 [−110, 50]	−12 [−83, 59]
Transformer	Delay	0.5	918	+6 [−60, 72]	+22 [−58, 102]
Transformer	Clean control	0	899	—	−64 [−187, 60]
Reactive	Noise	0	651	−16 [−338, 305]	−24 [−341, 293]
Reactive	Noise	0.5	651	+57 [−159, 274]	+125 [1, 250]
Reactive	Delay	0	588	−156 [−562, 251]	−100 [−393, 194]
Reactive	Delay	0.5	588	+64 [−211, 339]	+27 [−185, 239]
Reactive	Clean control	0	674	—	−98 [−376, 180]

~~The initial clean-condition means were 2,677.9 (transformer) and 3,328.1 (reactive) . The zero-step sensor conditions differed substantially. Mean noise-condition return was 2, 433.7 for the transformer~~

3.2 HalfCheetah

Table 3 lists the outcomes and Fig. 2 (top rows) shows their course. Before continued training, noise reduced the mean return of the transformer sources from 2,695 to 2,402 and that of the reactive sources from 3,239 to 2,900, whereas delay reduced it to 13 and −179. Continued clean training raised clean return by +782 and ~~2,875.5 for the reactive controller. Under the three-step delay, the corresponding means were −85.5 and −180.3. These are measurements after applying the sensor intervention and before continued training~~ +963, with every seed improving. The sources were therefore not converged, and the clean control, not $\Delta^A = 0$, is the reference for fault-specific forgetting. ~~Different initial performance and source-policy quality constrain cross-controller interpretation.~~

3.3 ~~Adaptation under Noise and Delay~~

~~Figure ?? shows the fixed-budget learning curves.~~

Adaptation. Under delay, the reactive controller's delayed-sensing return rose by +689 [373, 1,006] with every seed improving, to 510, still far below its clean

return. The transformer did not improve under delay (−115 [−259, 28]). Under noise, ~~mean return declined for both controllers with either replay strategy. Transformer returnchanged by −303.7 without retained replay and −328.7 with it. Reactive return changed by −505.8 and −148.2,respectively. Thus, continued training did not yield a mean noise-condition improvement within this budget, even though the reactive retained-replay arm declined less~~both controllers first lost return. The transformer recovered to +89 [−163, 341] by 20,000 steps, whereas the reactive controller stayed below its starting value (−443 [−546, −339], all seeds).

~~New-condition return during continued training. Lines are means over three source seeds; shaded bands are their minimum–maximum ranges, not confidence intervals. Small points at the final budget show individual seed means. Evaluation uses three deterministic episodes per seed and condition.~~

~~Delay produced positive mean changes but substantial variation.~~

Retention. Without retained replay, every fault branch ended below the clean control on clean data. The differences are −723 under noise and −3,278 under delay for the transformer, and −1,514 and −2,353 for the ~~transformer gained 98.1 return units and the~~ reactive controller~~gained 383.7. With retained replay, the corresponding gains were 10.2 and 228.2~~. All eight seeds agree in each case (Holm-adjusted $p = 0.0312$, the floor of this family). Relative to the sources, clean return fell under delay (−2,496 and −1,390) and, for the reactive controller, under noise (−550). The transformer's ~~no-replay seed-level changes ranged from −197.9 to +428.8; its retained-replay changes ranged from −38.4 to +105.4. Only two of three transformer seeds improved without replay~~ clean return under noise ended near its starting value (+59 [−104, 222]).

Retained replay. Retained replay reduced each clean-condition loss (Table 4). The paired effect on Δ^A was $D = +1{,}323$ for the transformer under delay, and $D = +1{,}406$ and $D = +394$ for the reactive controller under delay and noise, with all eight seeds positive in each case. With $\rho = 0.5$, the reactive controller's clean return under delay ended at its source value (+16 [−233, 266]), and its noise-condition decline also shrank ($D = +351$, all seeds). The cost appeared under delay. Replay reduced the reactive controller's delayed-sensing gain ($D = -284$, [−567, −1]; 6 of 8 seeds negative), the trade-off seen in the pilot. For the transformer, replay left the delayed-sensing outcome unchanged ($D = +46$, [−102, 195]) and changed neither outcome under noise. Each consistent effect has exact $p = 0.0078$ and ~~one of three improved with replay. All three reactive seeds improved under delayin both replay variants. These counts describe this pilot's realized seed outcomes, not estimated population success probabilities.~~ Holm-adjusted $p = 0.0625$, the floor for eight tests with eight seeds.

~~The trapezoidal mean return across the sampled learning curve was −27.8 without replay and −82.6 with replay for transformerdelay, compared with 51.2 and −89.8 for reactive delay. This summary includes intermediate deterioration that a final-point comparison would hide. It does not establish a recovery time because no recovery threshold was specified.~~

Table 4. Pre-specified paired replay effect $D = \text{metric}(\rho{=}0.5) - \text{metric}(\rho{=}0)$ at 20,000 steps (eight seeds). Holm-adjusted exact Wilcoxon p within each task's eight tests; +/− counts seeds with positive and negative D.

Controller	Fault	$D_{\Delta A}$ [95% CI]	p_{Holm}	+/−	$D_{\Delta B}$ [95% CI]	p_{Holm}	+/−
HalfCheetah-v5							
Transformer	Noise	0 [−158, 159]	1.00	5/3	+36 [−63, 134]	1.00	5/3
Transformer	Delay	+1,323 [595, 2,050]	0.0625	8/0	+46 [−102, 195]	1.00	5/3
Reactive	Noise	+394 [188, 599]	0.0625	8/0	+351 [153, 549]	0.0625	8/0
Reactive	Delay	+1,406 [816, 1,997]	0.0625	8/0	−284 [−567, −1]	0.31	2/6
Ant-v5							
Transformer	Noise	+7 [−21, 35]	1.00	4/4	+15 [−3, 32]	0.55	6/2
Transformer	Delay	+34 [−27, 96]	1.00	4/4	+36 [−31, 103]	1.00	5/3
Reactive	Noise	+149 [−116, 414]	1.00	6/2	+74 [−135, 282]	1.00	5/3
Reactive	Delay	+127 [−61, 315]	1.00	5/3	+220 [−34, 474]	0.31	7/1

3.3 ~~Retention and Paired Replay Effects~~

~~Every branch had a negative clean-condition change at the final budget (Fig. ??). For transformer delay, the mean loss was 825.4 without retained replay and 145.2 with replay. For reactive delay~~

Comparison with the pilot. The three-seed pilot had shown a small transformer gain under delay after 4,~~it was~~ 000 steps (+98). With eight seeds and five evaluation episodes, the change at that budget is −87 (6 of 8 seeds negative), and it remains negative through 20,000 steps. The reactive controller's delay gain and both controllers' clean-condition losses under delay persist and grow with the longer budget.

3.3 Ant

Sources. Under the same recipe, every Ant source learned to stand rather than walk. Across the 16 sources, the mean forward velocity in clean evaluation never exceeded 0.007 m/s in magnitude, every clean evaluation episode lasted 1,~~158.9 and 383.9. The corresponding noise-condition adaptation branches lost 320.9 versus 272.4~~ 000 steps, and the source returns (899 for the transformer and ~~641.2 versus 339.0~~ 674 for the reactive controller~~.~~

~~Clean-condition retention measured with the actual updated model. Zero is each branch's own initial clean-condition return. Negative values indicate loss of earlier measured performance. Lines and shaded ranges summarize three source seeds; the ranges are not confidence intervals.~~

~~Retained replay increased final clean-condition return relative to the paired no-replay branch in ten of twelve seed/controller/fault pairs. Its mean effect on clean return was positive in all four controller/fault comparisons. This was not uniform at the seed level: transformer seed 43~~) come mainly from Ant's healthy reward of one per step. Standing proved robust to the faults. Zero-step sensor-condition returns were 921 under noise and ~~seed 42 under delay retained less with replay. For both controllers, all three delay seeds ended with lower~~

~~delay-condition return under retained replay than under no replay. The paired comparison therefore exposes a cost in new-condition performance alongside improved average retention under delay. Neither armpreserved its own initial clean performance.~~ 918 under delay for the transformer, and 651 and 588 for the reactive controller, whose delayed-sensing episodes ended early in 12% of evaluations. Ant therefore tests retention during continued training more than adaptation.

~~Mean collection-and-optimization time per branch ranged from 55.1 to 75.0 seconds across transformer conditions and from 34.9 to 38.3 seconds across reactive conditions. Those times exclude evaluation and checkpoint I/O. They describe this implementation and machine load; they do not establish equal computational budgets or a generally faster adaptation algorithm~~

Continued training. In every arm, mean return on both conditions fell during the first 2,000 steps (Fig. 2, bottom rows), including the clean controls (-111 and -304), and then recovered partly or fully. Because the clean controls dipped as well, this transient comes from restarting training rather than from the faults; in HalfCheetah, the clean controls improved at every budget. At 20,000 steps, every 95% interval for the change in sensor-condition return included zero (Table 3). Continued clean training did not improve the standing sources: clean return changed by -64 $[-187, 60]$ for the transformer and -98 $[-376, 180]$ for the reactive controller. The fault-specific contrasts, all negative in HalfCheetah, were therefore positive or near zero. The transformer's noise branches ended above the clean control on clean data by $+90$ $[21, 159]$, with all eight seeds agreeing (Holm-adjusted $p = 0.0312$, the floor); the other three intervals included zero.

4 ~~Discussion~~

~~The pilot makes the distinction between adaptation and retention observable. A controller can gainreturn under delayed sensing while losing performance with clean observations. Under delay, mixing earlier experience into the update batch reduced the average clean-condition loss for both controllers, but also reduced their mean new-condition gain. The tested replay mixture therefore trades one measured outcome against the other; it does not satisfy the original objective of rapid adaptation with preserved earlier behavior.~~

Retained replay. All eight paired replay effects had positive means, but every t interval included zero (Table 4). The most consistent was the effect on the reactive controller's delayed-sensing return ($D = +220$, $[-34, 474]$; 7 of 8 seeds positive; Holm-adjusted $p = 0.31$). Without replay, that return changed by -156 and 30% of delayed-sensing episodes ended early at 20,000 steps; with replay, 10% did. This effect has the opposite sign to HalfCheetah's, where replay reduced the delayed-sensing gain. With replay, the reactive controller's clean return under noise rose by $+125$ $[1, 250]$, with all eight seeds improving.

~~Noise and delay also require separate interpretation. The source policies already obtained comparatively high returns under the chosen noise level, whereas~~

Table 5. Critic diagnostic (mean over eight seeds) at the source checkpoint and after 20,000 steps with $\rho = 0$ or 0.5: root-mean-square held-out soft Bellman residual on clean (A) and corrupted (B) evaluation data, and the value-bias gap, the Monte-Carlo bias on B data minus that on A data of the same checkpoint (positive: values on corrupted data are overestimated relative to clean data; exploratory).

		Residual, clean data A			Residual, corrupted data B			Value-bias gap $B - A$		
Controller	Fault	Source	ρ=0	ρ=0.5	Source	ρ=0	ρ=0.5	Source	ρ=0	ρ=0.5
HalfCheetah-v5										
Transformer	Noise	0.58	0.58	0.61	4.25	1.13	1.49	+27	+18	+13
Transformer	Delay	0.58	1.61	2.87	4.36	0.96	1.38	+171	+27	+89
Reactive	Noise	1.55	1.04	1.72	6.89	1.81	3.40	+28	+19	+23
Reactive	Delay	1.55	3.07	2.73	5.84	1.91	3.61	+268	+114	+204
Ant-v5										
Transformer	Noise	2.16	0.23	0.30	3.48	0.44	0.58	−2	0	−2
Transformer	Delay	2.16	1.35	0.34	1.16	0.27	0.28	−1	0	−1
Reactive	Noise	1.06	0.28	0.22	5.56	1.00	0.97	+2	+2	+8
Reactive	Delay	1.06	0.72	3.05	2.00	3.03	1.23	0	+15	+3

~~delay caused much lower initial returns. Further training reduced average noise-condition return, so more interaction and gradient updates were not sufficient to improve that condition within this budget. Under delay there was more room for improvement, yet transformeroutcomes differed in sign across seeds~~

3.1 Critic Diagnostic

Table ?? reports the diagnostic, which is not part of the pre-specified analysis; the value-bias gap is exploratory. At the HalfCheetah sources, the held-out soft Bellman residual on corrupted data was several times that on clean data, and continued training on corrupted data reduced it in every arm. Retained replay raised the reactive controller's residual on corrupted data under both faults (by +1.59 under noise and +1.69 under delay, all eight seeds each; Holm-adjusted $p = 0.0625$, the floor) and its clean-data residual under noise (+0.68, all seeds). For the transformer, only the increase on noisy data had an interval excluding zero (+0.36 [0.02, 0.69]). In Ant, replay raised neither residual consistently; it lowered the transformer's clean-data residual under delay (−1.01, all eight seeds).

The value-bias gap separates the faults. ~~Pooling the faultsinto one headline score would obscure these distinctions.~~

~~The comparison does not demonstrate that history is inherently ineffective. It compares particular pretrained controllers whose capacities, representations,~~ The HalfCheetah source critics overvalued delayed-sensing histories relative to clean ones by +171 (transformer) and +268 (reactive), with all eight seeds positive in both, against +27 and +28 under noise. Training on delayed data alone shrank this gap to +27 and ~~initial outcomes differ. Aligning context support removes a known implementation inconsistency, but it cannot by itself ensure a useful learned representation or stable continued learning. The reactive controller's largermean delay gain is evidence about these tested configurations, not a controlled~~

Table 6. Per-action inference latency (ms) of the HalfCheetah source controllers (seed 42), batch size one, real model inputs, simulation excluded; 10,000 timed steps after warm-up on an AMD Ryzen 9 8940HX CPU and a GeForce RTX 5070 Laptop GPU. The transformer recomputes its full 32-token context at every step.

Controller	Device	Mean	p95	p99	Max
Transformer	CPU, 1 thread	0.58	0.62	0.66	1.56
Transformer	CPU, 4 threads	0.58	0.62	0.73	1.46
Transformer	GPU	1.64	1.72	1.79	3.03
Reactive	CPU, 1 thread	0.09	0.11	0.13	0.53
Reactive	CPU, 4 threads	0.09	0.11	0.13	0.48
Reactive	GPU	0.48	0.53	0.56	0.94

causal estimate of the value of memory +114. With retained replay it stayed larger, at +89 and +204; paired effects of +62 [16, 109] and +90 [41, 139] (7 and 7 of 8 seeds positive). Under noise, replay changed the gap by only −6 (transformer) and +4 (reactive). In Ant, every mean gap stayed within 16 return units of zero.

Likewise, the observed clean-condition declines should not be assigned to a single mechanism. They occur after training on changed sensing, but optimizer resets, source-policy quality, representation updates, and the narrow retained pool may also influence the result. A continued-clean-training control is needed to quantify deterioration caused by continued optimization alone. Primacy bias or loss of plasticity remain possible topics for subsequent diagnosis, rather than explanations established by these measurements. The practical outcome of this pilot is a verified experimental basis for that diagnosis and a clear requirement to evaluate old and new conditions together.

3.2 Limits of the Pilot

The source checkpoints were trained with actuator perturbations,

3.2 Inference Latency

Table ?? lists the per-action latency. On one CPU thread the transformer took 0.58 ms on average and 0.66 ms at the 99th percentile, and no timed CPU call exceeded 1.56 ms. Every CPU call therefore fit the 2 ms period of 500 Hz control, and the clean condition is an evaluation setting rather than a newly optimized clean-only task. Adapting these models may differ from training 10 ms period of 100 Hz control by a wide margin; the reactive controller took 0.09 ms. Four threads brought no gain. At batch size one the GPU was slower than the CPU (1.64 ms mean and 3.03 ms maximum for the transformer), consistent with per-call launch and transfer overhead dominating such small networks. The figures cover the network call on a laptop workstation, not a complete control loop or an embedded processor.

4 Discussion

Adaptation and retention are separate outcomes. Measuring both on the same checkpoints separates patterns that a single score would merge. Under delay in HalfCheetah, the reactive controller adapted and lost clean performance; retained replay preserved its clean performance and kept part of the gain. The transformer lost clean performance without adapting. Under noise, the transformer kept both returns near their starting values, whereas the reactive controller lost both, far less so when half of each batch came from earlier clean experience. In HalfCheetah, every fault branch without replay also forfeited the improvement that continued clean training produced, so a branch that merely holds its starting return has still fallen behind. Ant shows the control's value in the other direction: its standing sources lost little to either fault and did not improve under clean training, so changes in its fault branches reflect continued training more than the faults.

Why the controllers differ. The information available to each controller predicts an advantage for the transformer under both faults. Its context holds the last three commands needed to reconstruct the current state from the delayed reading, and a window of noisy readings that it could average; the reactive token holds one reading and the previous command. In HalfCheetah, the noise results agree with this prediction, but the delay results contradict it: only the reactive controller improved. We see four candidate explanations, none tested here. First, compensating for the delay requires the encoder to learn a new computation, and it learns only through critic regression, because the actor uses detached features. Second, continued training is relatively weaker for the transformer: each update trains 64 transitions, a ~~sensor-aware agent from scratch. Optimizers and entropy temperature are reinitialized; this may itself influence continued learning. The current pilot cannot separate that effect from sensitivity to~~ sixteenth of the ~~new observation distribution. Both replay variants share that initialization, which supports their within-model comparison but does not remove the limitation.~~

~~The retained pool contains only two deterministic clean episodes per model. It covers a narrow part of the old state distribution and contains no explicit behavioral-cloning or feature-preservation loss. Half of a batch drawn from this pool is a particular rehearsal strategy, not a general guarantee of retention. The two architectures also~~ 1,024 per critic update in its source training, whereas the reactive controller trains on 64 transitions in both phases. Third, 5,000 updates at learning rate 10^{-4} may be too few to reorganize a representation that is adequate for clean sensing. Fourth, the controllers differ in capacity ~~, input processing, and pretraining outcomes. Three seeds and three fixed evaluation episodes per condition provide a limited basis for generalization. Averages may conceal opposite outcomes across source models; the released tables retain the seed-level values.~~ and initial returns (Table 1). Separating these explanations requires matched ablations, such as a transformer restricted to one token or a reactive controller given stacked readings and commands.

~~This is a single locomotion environment with two fixed sensor conditions, one delay, one noise vector, and a 4,000-step adaptation budget. It does not measure arbitrary new-map transfer, visual sensing, unknown fault onset during deployment, changing severity, or long sequences of tasks. The model retains a current reward and command channel, so its information loss is narrower than complete sensor failure. No longer-budget plateau, recovery threshold, or retention tolerance is inferred from the three recorded evaluation budgets. We report realized pilot behavior rather than declare that fast adaptation without forgetting has been achieved.~~

Replay mixing. Of the sixteen paired replay effects in the two tasks, only one had a negative mean: the HalfCheetah reactive controller's smaller delayed-sensing gain. The critic diagnostic is consistent with the target blending described in Sec. 2.5. The source critics overvalued delayed histories relative to clean ones, anticipating only part of the return that the source policies lost under delay. Training on delayed data alone removed most of this value-bias gap; with clean experience mixed in, the gap stayed larger, and the reactive controller's residual on delayed data stayed higher. The transformer showed the same gap pattern but had no delayed-sensing gain to lose. Under noise, replay improved both of the reactive controller's returns while raising its residuals, so a larger held-out residual alone does not indicate a worse policy. In Ant, where the faults barely mattered, the gaps stayed near zero. A two-episode clean pool is a minimal form of rehearsal. In HalfCheetah, retained replay did not match the clean control's gains on clean data, and behavior-cloning or feature-preservation terms [15,18] could act differently.

4.1 ~~Next Evaluation~~

~~A subsequent study should specify acceptable old-task degradation and a new-task recovery criterion before final evaluation. It should include more independent source models and evaluation states,a longer budget, multiple task orders, and a baseline that continues training on A for the same number of updates. That continuation control would help distinguish general fine-tuning instability from changes specific to B. A matched context encoder and parameter budget would improve architectural attribution.~~

~~The retention mechanism also warrants a controlled comparison: ordinary replay, retained replay,~~

Limitations. The sources were trained with actuator perturbations and then warm-started; sources trained to convergence, or with sensor faults, may behave differently. The Ant sources stood rather than walked, so Ant tests the retention of a posture rather than of locomotion; we did not tune the source recipe after seeing this. Twenty thousand steps are 5% of the source recipe's 400,000-step budget, and ~~an explicitly defined behavioral or feature anchor should share the same new-data budget. Data selection and source-policy quality should be varied separately. Any resulting candidate must still be evaluated on the earlier~~

~~conditions using its actual updated weights. These additions would move the evaluation closer to the original goal while preserving the distinction between learning a new condition~~ 5,~~retaining an old one, and remaining capable of future learning~~000 updates are 5% of its critic updates, so slower adaptation could still occur. We used one severity per fault, one learning-rate setting and two replay fractions, without tuning. Humanoid-v5 was not run: SAC's reported Humanoid learning curves span millions of steps [8], far beyond the 400,000-step source recipe, and our single workstation was at its memory limit. With eight seeds, exact tests cannot reach a family-wise 0.05 level in the primary family, so we rely on interval estimates. The critic diagnostic lies outside the pre-specified analysis, and the value-bias gap is exploratory. Evaluations use five deterministic episodes per condition and budget, and the controllers' differences in capacity prevent attributing their contrast to memory alone.

5 Conclusion

We ~~implemented actual sensor noise and observation delay, aligned transformer history support, and completed a paired pilot of~~measured adaptation and retention during continued SAC training ~~with explicit retention measurements. Across three pretrained~~ under sensor faults, with eight seeds per controller, ~~retained replay reduced average~~ two tasks, a clean-continuation control and pre-specified paired tests. In HalfCheetah, delay separated the two outcomes: the reactive controller adapted while losing clean performance, and the transformer lost clean performance without adapting. Retaining half a batch of earlier clean experience reduced every clean-condition ~~loss in every controller/fault comparison but did not eliminate it. Under delay , improved average retention accompanied smaller new-condition gains; under noise, mean new-condition performance declined in all variants. These findings do not establish fast adaptation without forgetting. They identify the tradeoff exhibited by this configuration and motivate longer, controlled studies of replay coverage,~~ loss, at the cost of part of the reactive controller's delay gain, and the critic diagnostic is consistent with values blended across the two sensing channels. In Ant, standing sources tolerated both faults, so that task probed continued training more than adaptation. Next steps are matched ablations that separate memory from capacity, explicit retention objectives, and ~~continued-learning stability~~source recipes that produce locomotion.

~~AI Usage Declaration~~

~~Google Antigravity with a Geminimodel assisted implementation of earlier research architecture and algorithms~~

Code and data availability. Code, sensor-fault wrappers, protocols, evaluation records, and checkpoints: https://github.com/Fane-Nathan/tmrl.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

AI Usage Declaration. Google Antigravity (Gemini) assisted earlier implementation. OpenAI Codex assisted manuscript restructuring, primary-source lookup, ~~formatting,~~ code inspection, ~~implementation and testing of the~~ and sensor-fault pilot ~~, experiment orchestration, result verification, and preparation of analysis and plotting scripts. The reported~~ testing. Claude Code (Anthropic) assisted the camera-ready revision: extending the experiment runner to more seeds, longer budgets, the clean control and Ant-v5, statistical and latency scripts, and manuscript revision. All observations were generated by executed simulator runs~~; tables and figures~~ , and tables and result plots are computed from saved run records. Generative AI was not used to invent ~~experimental observations~~ observations or references. The authors remain fully responsible for the implementation, ~~source~~ verification, analysis, interpretation ~~, and~~ and final manuscript.

Author Contributions. F.S.: conceptualization, sensor-fault wrappers, adaptation pipeline, experiments, data analysis, and manuscript drafting. K.R.: software verification, statistical analysis, auditing, and figures. D.: baseline reproduction, replay diagnostics, and latency benchmarking. S.P.: supervision, methodology validation, project administration, and review. All authors approved the final manuscript.

References

1. Ball, P.J., Smith, L., Kostrikov, I., Levine, S.: Efficient online reinforcement learning with offline data. In: Proceedings of the 40th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 202, pp. 1577–1594 (2023), https://proceedings.mlr.press/v202/ball23a.html
2. Bhatt, A., Palenicek, D., Belousov, B., Argus, M., Amiranashvili, A., Brox, T., Peters, J.: CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In: International Conference on Learning Representations (2024), https://iclr.cc/virtual/2024/poster/18699
3. Chen, X., Wang, C., Zhou, Z., Ross, K.: Randomized ensembled double Q-learning: Learning fast without a model. In: International Conference on Learning Representations (2021), https://arxiv.org/abs/2101.05982
4. Dohare, S., Hernandez-Garcia, J.F., Lan, Q., Rahman, P., Mahmood, A.R., Sutton, R.S.: Loss of plasticity in deep continual learning. Nature **632**, 768–774 (2024). https://doi.org/10.1038/s41586-024-07711-7
5. Duan, Y., Schulman, J., Chen, X., Bartlett, P.L., Sutskever, I., Abbeel, P.: RL^2: Fast reinforcement learning via slow reinforcement learning. arXiv:1611.02779 (2016). https://doi.org/10.48550/arXiv.1611.02779
6. Grigsby, J., Fan, J., Zhu, Y.: AMAGO: Scalable in-context reinforcement learning for adaptive agents. In: International Conference on Learning Representations (2024), https://proceedings.iclr.cc/paper_files/paper/2024/hash/7204434dcb9383a1454dc1e97e58ea9c-Abstract-Conference.html
7. Grigsby, J., Sasek, J., Parajuli, S., Adebi, D., Zhang, A., Zhu, Y.: AMAGO-2: Breaking the multi-task barrier in meta-reinforcement learning with transformers. In: Advances in Neural Information Processing Systems. vol. 37 (2024). https://doi.org/10.52202/079017-2776
8. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: Proceedings of the 35th International Conference on Machine Learning.

Proceedings of Machine Learning Research, vol. 80, pp. 1861–1870 (2018), https://proceedings.mlr.press/v80/haarnoja18b.html
9. Hiraoka, T., Imagawa, T., Hashimoto, T., Onishi, T., Tsuruoka, Y.: Dropout Q-functions for doubly efficient reinforcement learning. In: International Conference on Learning Representations (2022), https://iclr.cc/virtual/2022/poster/6233
10. Jordan, S.M., White, A., da Silva, B.C., White, M., Thomas, P.S.: Position: Benchmarking is limited in reinforcement learning research. In: Proceedings of the 41st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 235, pp. 22551–22569 (2024), https://proceedings.mlr.press/v235/jordan24a.html
11. Laskin, M., Wang, L., Oh, J., Parisotto, E., Spencer, S., Steigerwald, R., Strouse, D., Hansen, S., Filos, A., Brooks, E., Gazeau, M., Sahni, H., Singh, S., Mnih, V.: In-context reinforcement learning with algorithm distillation. In: International Conference on Learning Representations (2023), https://iclr.cc/virtual/2023/oral/12612
12. Ni, T., Eysenbach, B., Salakhutdinov, R.: Recurrent model-free RL can be a strong baseline for many POMDPs. In: Proceedings of the 39th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 162, pp. 16691–16723 (2022), https://proceedings.mlr.press/v162/ni22a.html
13. Nikishin, E., Schwarzer, M., D'Oro, P., Bacon, P.L., Courville, A.: The primacy bias in deep reinforcement learning. In: Proceedings of the 39th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 162, pp. 16828–16847 (2022), https://proceedings.mlr.press/v162/nikishin22a.html
14. Patterson, A., Neumann, S., White, M., White, A.: Empirical design in reinforcement learning. Journal of Machine Learning Research **25**(318), 1–63 (2024), https://jmlr.org/papers/v25/23-0183.html
15. Rolnick, D., Ahuja, A., Schwarz, J., Lillicrap, T., Wayne, G.: Experience replay for continual learning. In: Advances in Neural Information Processing Systems, vol. 32 (2019), https://papers.nips.cc/paper_files/paper/2019/hash/fa7cdfad1a5aaf8370ebeda47a1ff1c3-Abstract.html
16. Towers, M., Kwiatkowski, A., Terry, J., Balis, J.U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J.J., Tan, H., Younis, O.G.: Gymnasium: A standard interface for reinforcement learning environments. arXiv:2407.17032v3 (2024), https://arxiv.org/abs/2407.17032v3
17. Wang, W., Han, D., Luo, X., Li, D.: Addressing signal delay in deep reinforcement learning. In: International Conference on Learning Representations (2024), https://proceedings.iclr.cc/paper_files/paper/2024/hash/45cc967d7899616f51993b7b363d35b5-Abstract-Conference.html
18. Wołczyk, M., Cupiał, B., Ostaszewski, M., Bortkiewicz, M., Zając, M., Pascanu, R., Kuciński, Ł., Miłoś, P.: Fine-tuning reinforcement learning models is secretly a forgetting mitigation problem. In: Proceedings of the 41st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 235, pp. 53039–53078 (2024), https://proceedings.mlr.press/v235/wolczyk24a.html